

CENG 476 - Introduction to Deep Learning

Squat Posture Classification

Custom CNN, MobileNetV3 Transfer Learning, and Fine-Tuning

Group Members: Melih Berk Ürek

Student IDs: 220444408

Summer Term 2025-2026

Date: 28 August 2026

1. Introduction

1.1 Problem Definition

This project addresses a three-class, single-label image classification problem for squat posture analysis. Given a squat image, the system predicts one of three labels: Good, Bad back, or Bad heel. The practical goal is to investigate whether a deep learning model can distinguish correct squat posture from two common posture-error categories using only RGB images.

1.2 Motivation and Project Objective

The project was designed not only to maximize training accuracy, but to study generalization to unseen data. This became especially important because many samples originate from consecutive video frames: the same person, clothes, room, and background may appear in many visually similar images. A naive frame-level random split can therefore make validation performance unrealistically optimistic. The project evolved from a custom CNN baseline to transfer learning with MobileNetV3-Large and finally to fine-tuning.

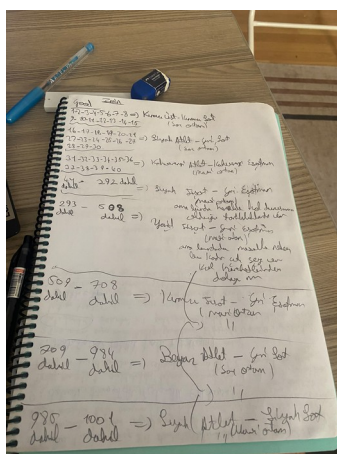
1.3 Dataset

Split	Images	Bad back	Bad heel	Good
Training	2,274	838	708	728
Validation	563	146	144	273
Test	969	338	321	310

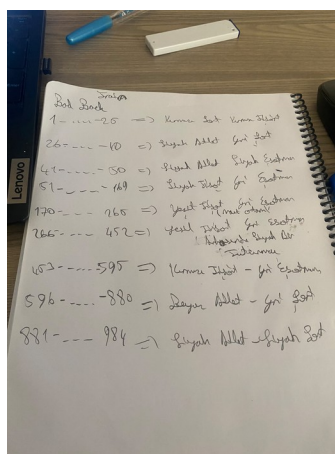
The validation set was manually separated using visually related frame ranges, and an index-overlap check reported no overlap between training and validation indices. The independent test set was not used for model selection. This manual group-based validation strategy was introduced to reduce leakage from near-duplicate consecutive frames.

1.4 Manual Validation Split Strategy

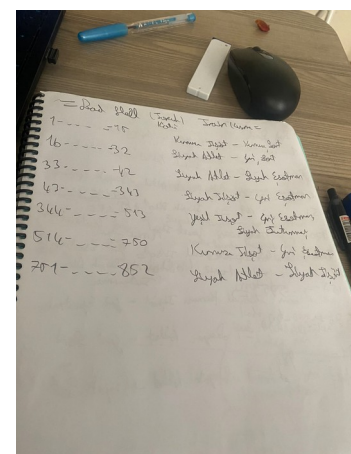
Because the dataset contains consecutive video frames, the original recordings were inspected manually before the final split. Frame ranges with similar visual conditions (for example, the same subject, clothing, and recording environment) were identified for each class. Representative handwritten grouping notes are shown below. These notes document the practical grouping step used before selecting the validation ranges.



Good



Bad back



Bad heel

Figure 1. Manual inspection notes used to group consecutive frames for Good, Bad back, and Bad heel classes.

Final validation ranges used in the training pipeline:

Class	Validation frame ranges
Good	1-15, 16-30, 31-40, 293-508, 985-1001
Bad heel	1-15, 16-32, 33-42, 751-852
Bad back	1-25, 26-40, 41-50, 170-265

The purpose of this strategy was to avoid a purely image-level random split in which neighboring, nearly identical frames could appear in both training and validation sets. After the split, train/validation index overlap was checked and no overlap was found. The resulting validation set contained 563 images.

2. Related Work and Baseline Strategy

Image classification is commonly approached with convolutional neural networks (CNNs). In this project, a custom CNN trained from scratch was used as the baseline so that the effect of transfer learning could be measured. The second architecture was MobileNetV3-Large with pretrained ImageNet weights.

MobileNetV3 was selected because it provides pretrained visual features while remaining computationally efficient. Compared with ResNet-style networks, MobileNetV3 emphasizes lightweight operations such as depthwise separable convolutions, whereas ResNet is built around residual/skip connections and is generally heavier. This made MobileNetV3 a suitable candidate for a posture system that could later be considered for real-time or embedded deployment.

For a balanced three-class task, random guessing would be approximately 33.3% accuracy. The custom CNN result was therefore used as the practical learned baseline, while MobileNetV3 experiments tested whether pretrained features could improve performance on the limited and visually correlated dataset.

3. Proposed Method

3.1 Preprocessing and Data Augmentation

All images were processed at 224x224 resolution. Training images received stochastic augmentation, while validation and test images used deterministic resizing and ImageNet normalization only. This separation ensures that evaluation is performed on stable inputs while training receives additional visual diversity.

Operation	Project Setting	Purpose
RandomResizedCrop	224, scale=(0.7, 1.0)	Changes crop/scale so the model is less dependent on exact framing.
RandomHorizontalFlip	p=0.5	Reduces dependence on left/right orientation.
RandomRotation	+/-20 degrees	Simulates moderate camera or body-angle variation.
ColorJitter	brightness=0.3, contrast=0.3, saturation=0.2	Simulates illumination and color variation.
RandomErasing	p=0.25, scale=(0.02, 0.2)	Reduces dependence on a single local visual cue.
Normalization	mean=[0.485,0.456,0.406], std=[0.229,0.224,0.225]	Matches standard ImageNet input statistics.

Data augmentation does not create genuinely new subjects or new motion recordings; it creates plausible variations of existing images. For this reason it can reduce memorization, but it cannot fully replace real data diversity.

3.2 Custom CNN

The custom model is a four-stage convolutional baseline. Its feature extractor follows the pattern Conv2d -> BatchNorm2d -> ReLU -> MaxPool, with channel depth increasing 32 -> 64 -> 128 -> 256. The classifier maps the extracted representation to three output logits and uses dropout for regularization. Batch normalization stabilizes intermediate activations, ReLU introduces non-linearity, and max pooling progressively reduces spatial resolution.

During project development, regularization was strengthened after overfitting was observed. Earlier experiments used Dropout=0.20 and weight decay=1e-3; in the final Custom CNN code, dropout was increased to 0.60, weight decay to 1e-2, and label smoothing=0.10 was used. These changes were intended to reduce co-adaptation, discourage very large weights, and prevent overconfident class predictions.

3.3 MobileNetV3 Transfer Learning

The transfer-learning stage used MobileNetV3-Large with pretrained ImageNet weights. Initially, the feature extractor was frozen and the classifier was adapted to the three squat-posture classes. The final MobileNetV3 code used Linear -> Hardswish -> Dropout(0.40) -> Linear(3), CrossEntropyLoss with label smoothing=0.05, AdamW with learning rate 1e-3 and weight decay 1e-3, and CosineAnnealingLR with T_max=30. MobileNetV3 used the same final training DataLoader and augmentation pipeline defined for the manual split.

The reason for transfer learning is that a pretrained network already contains useful low- and mid-level visual representations such as edges, textures, and shapes. With a relatively small task-specific dataset, reusing these features can be more effective than learning every representation from scratch.

3.4 Fine-Tuning

After the initial MobileNetV3 stage, the last four feature blocks were unfrozen. Fine-tuning used different learning rates for different parts of the network: 1e-5 for the pretrained backbone and 1e-4 for the classifier. The smaller backbone learning rate was chosen to adapt pretrained features gradually without destroying useful ImageNet knowledge. Early stopping with patience=5 and best-validation-loss checkpointing were used.

3.5 Activation and Output Design

The custom CNN uses ReLU in hidden layers. MobileNetV3 uses its architecture-specific nonlinearities, including Hardswish in the adapted classifier used in the transfer-learning experiment. The final layer produces three raw logits. CrossEntropyLoss is applied directly to these logits; therefore an explicit Softmax layer is not required during training. Softmax is only needed when probabilities are displayed during inference.

4. Experimental Setup

4.1 Reproducibility and Data Loading

The final project code fixed the PyTorch and NumPy random seeds to 42 to make random operations more reproducible. In the manual-split pipeline, DataLoader used batch_size=32 and num_workers=0. Training batches were shuffled, whereas validation and test batches were not shuffled. Batch size controls how many samples contribute to one optimization step, while num_workers controls parallel CPU data-loading processes and is not a model parameter.

4.2 Optimization and Regularization

Stage	Key Training Settings
Custom CNN	AdamW; lr=1e-4, weight_decay=1e-2, label_smoothing=0.10, dropout=0.60; CosineAnnealingLR (T_max=30); maximum 30 epochs; patience=5.
MobileNetV3 transfer learning	AdamW; classifier lr=1e-3, weight_decay=1e-3, label_smoothing=0.05, dropout=0.40; CosineAnnealingLR (T_max=30); maximum 30 epochs; patience=5.
Fine-tuned MobileNetV3	Last 4 feature blocks unfrozen; backbone lr=1e-5; classifier lr=1e-4; weight_decay=1e-4; label_smoothing=0.05; CosineAnnealingLR (T_max=20); maximum 20 epochs; patience=5.

AdamW was selected because it combines adaptive optimization with decoupled weight decay. Weight decay, dropout, label smoothing, augmentation, early stopping, and transfer learning were all used as complementary defenses against overfitting rather than relying on a single technique.

4.3 Early Stopping and Model Selection

Validation loss was the main model-selection criterion. Whenever validation loss improved, the corresponding weights were saved as the best checkpoint. If validation loss failed to improve for five consecutive epochs, training stopped. A patience of five provides a compromise: patience=1 could stop because of a single noisy epoch, while a very large patience would continue training unnecessarily after generalization had stopped improving.

5. Results and Discussion

5.1 Final Model Comparison

Model	Final Test Accuracy	Change
Custom CNN	35.50%	Baseline
MobileNetV3	48.81%	+13.31 percentage points
Fine-Tuned MobileNetV3	50.57%	+1.76 points vs. MobileNetV3; best result

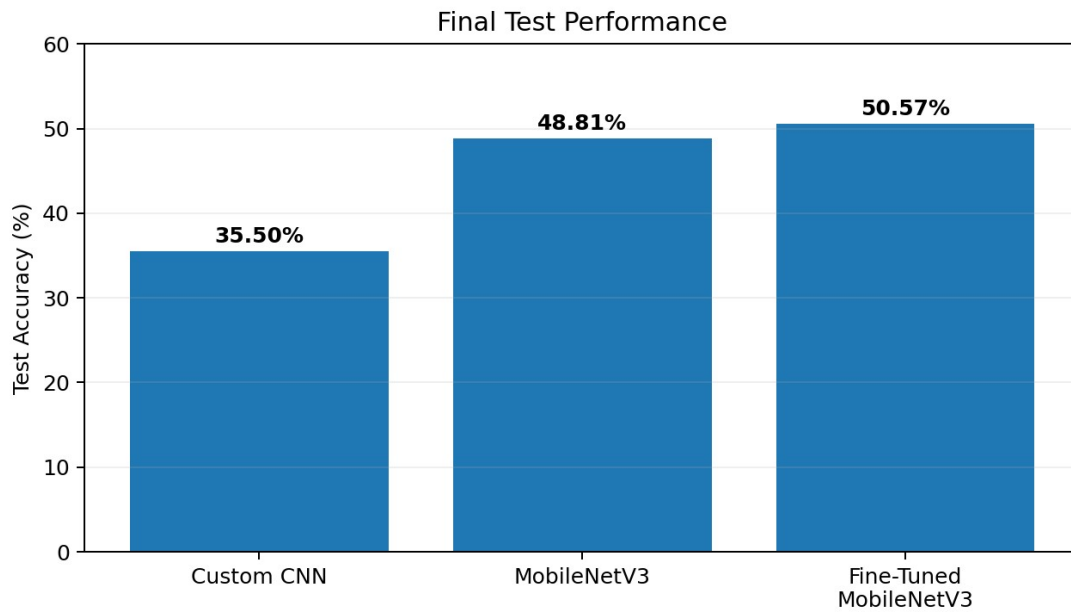


Figure 1. Final test accuracy comparison.

5.2 Custom CNN Results

With the manual validation split, the custom CNN reached its best validation loss at epoch 1 (1.3598) and achieved 35.50% final test accuracy. The training accuracy continued to increase while validation performance remained unstable, which is a strong overfitting signal.

Class	Precision	Recall	F1-score	Support
Bad back	0.35	0.29	0.32	338
Bad heel	0.36	0.76	0.49	321
Good	0.00	0.00	0.00	310
Macro avg	0.24	0.35	0.27	969
Weighted avg	0.24	0.36	0.27	969

The most important class-level failure was the Good class, for which recall was 0.00 in the reported final test. This shows that overall accuracy alone is insufficient and that class-wise metrics are necessary.

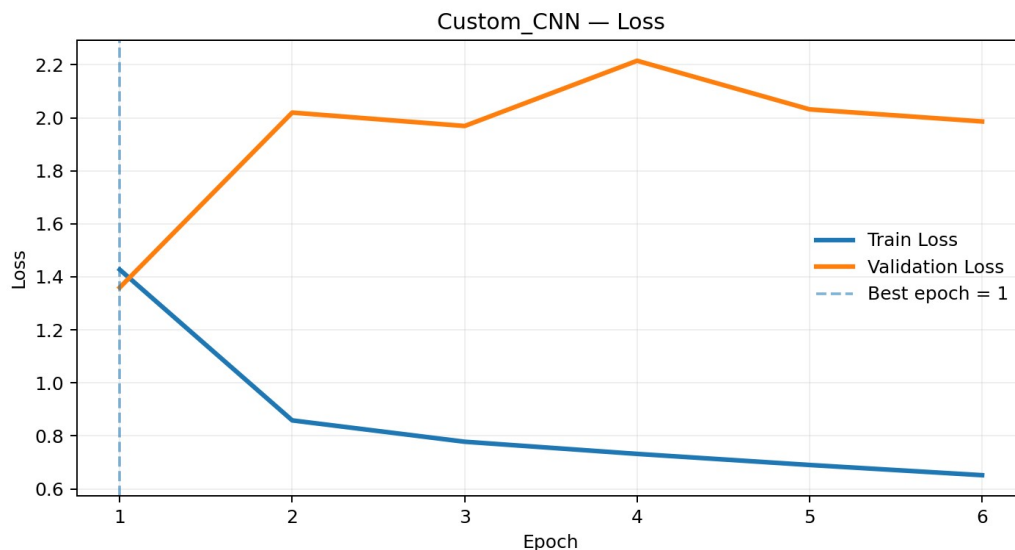


Figure 2. Custom CNN training and validation loss.

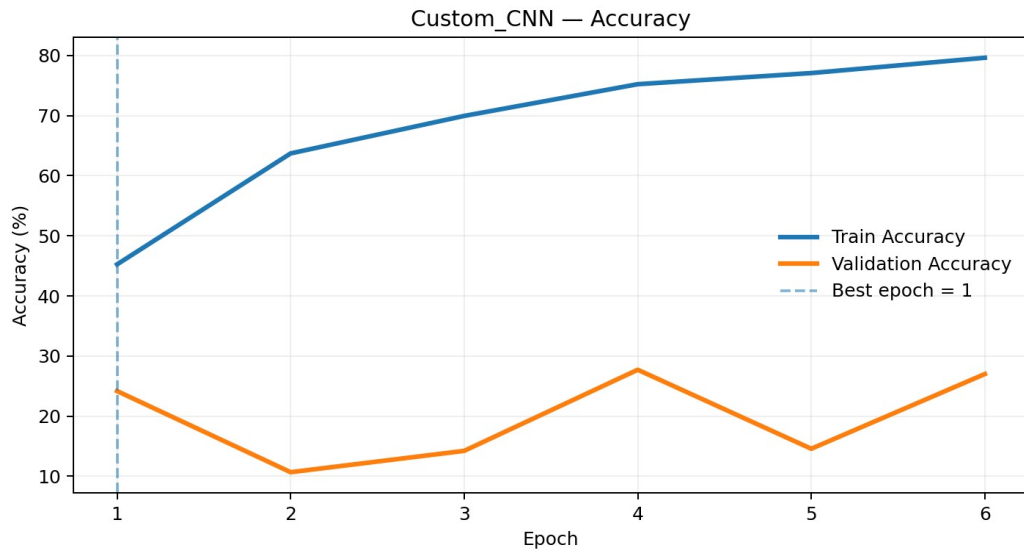


Figure 3. Custom CNN training and validation accuracy.

5.3 MobileNetV3 Results

The initial MobileNetV3 transfer-learning experiment achieved 48.81% final test accuracy. The best checkpoint was selected at epoch 9 with validation loss 1.7915. Training accuracy reached approximately 96%, while validation accuracy was around 18% at the best-loss region, producing a very large train-validation gap.

Class	Precision	Recall	F1-score	Support
Bad back	0.41	0.99	0.58	338
Bad heel	0.97	0.19	0.31	321
Good	0.86	0.26	0.40	310
Macro avg	0.75	0.48	0.43	969
Weighted avg	0.74	0.49	0.43	969

Bad back recall was extremely high (0.99), while Bad heel and Good recall were much lower. This indicates a strong class bias: the model often recognized Bad back but did not separate all three classes consistently.

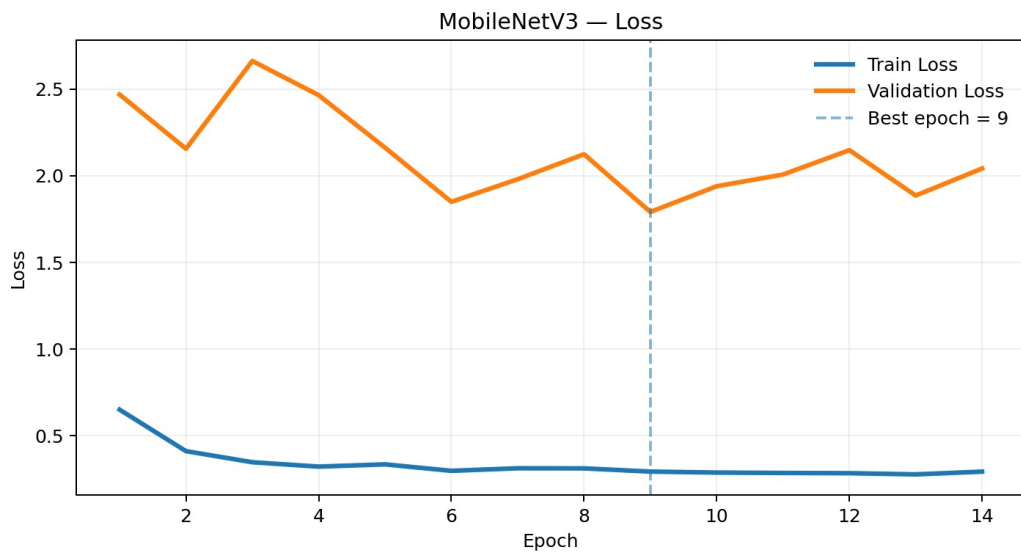


Figure 4. MobileNetV3 training and validation loss.

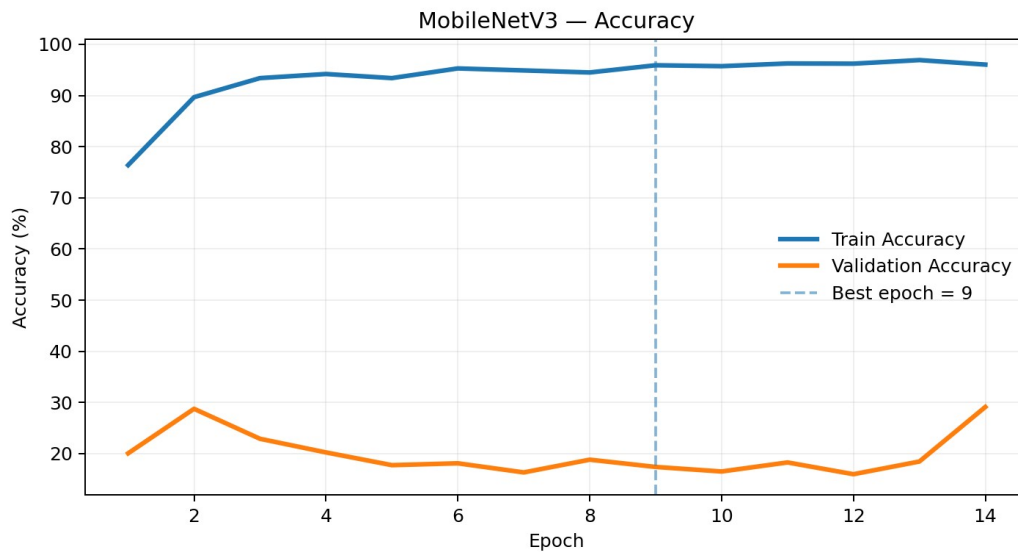


Figure 5. MobileNetV3 training and validation accuracy.

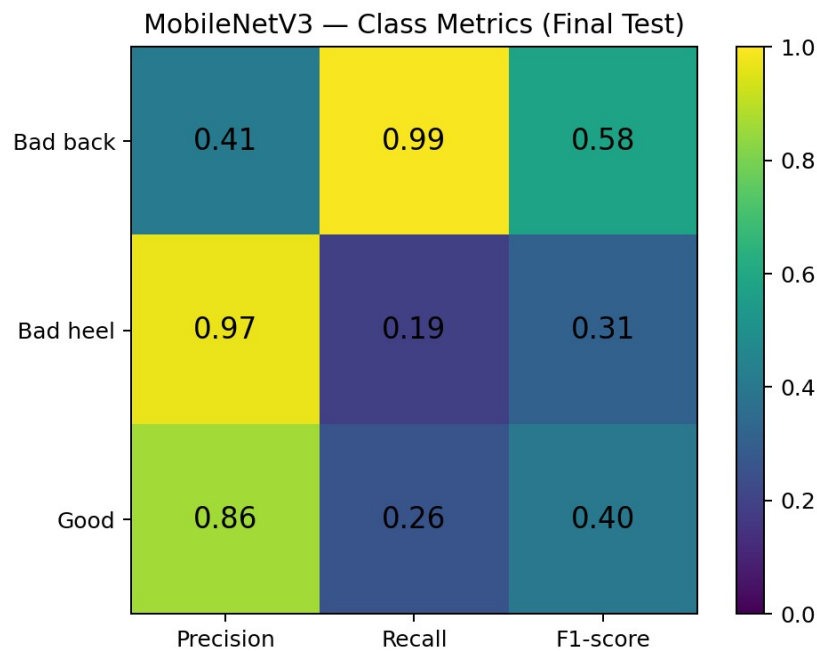


Figure 6. MobileNetV3 class-level metrics.

5.4 Fine-Tuned MobileNetV3 Results

Fine-tuning produced the best final result. The best checkpoint occurred at epoch 12 with validation loss 1.7409, and final test accuracy increased to 50.57%. This is a gain of 1.76 percentage points over the frozen-backbone MobileNetV3 experiment. However, training accuracy approached 99% while validation accuracy remained around 22%, so fine-tuning improved the final test result without eliminating the generalization problem.

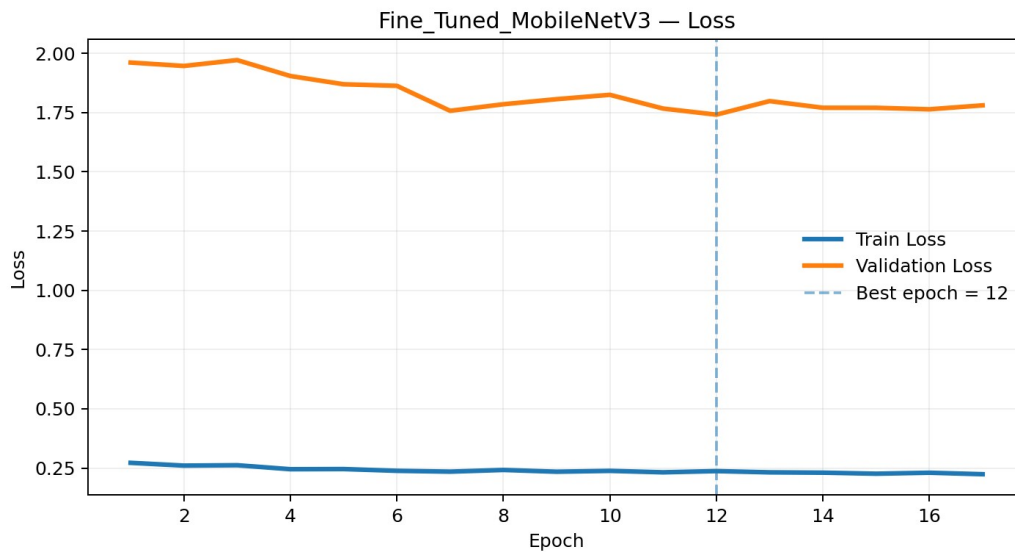


Figure 7. Fine-tuned MobileNetV3 training and validation loss.

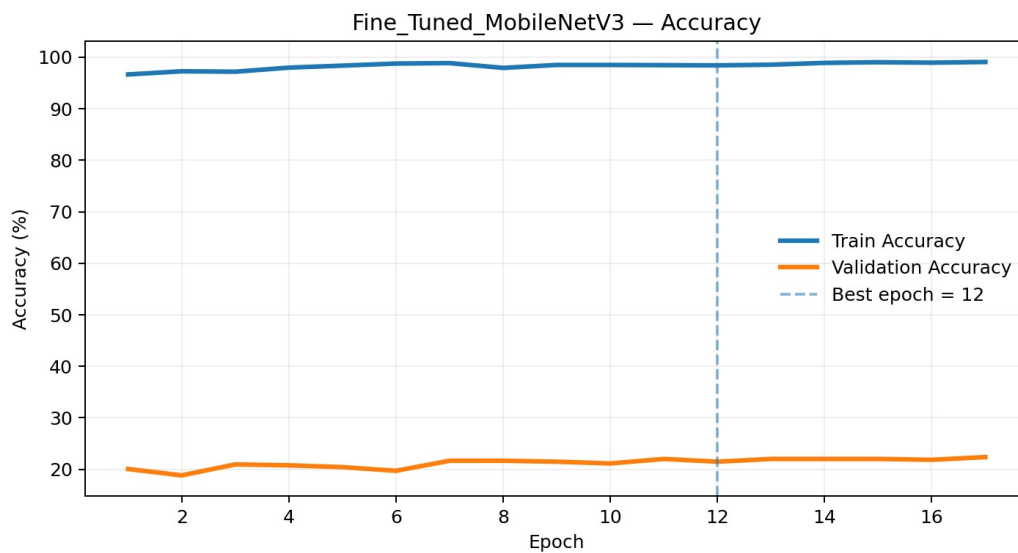


Figure 8. Fine-tuned MobileNetV3 training and validation accuracy.

5.5 Overfitting and Error Analysis

Overfitting was the central difficulty of the project. Strong training performance did not translate into equally strong validation performance. Several defenses were applied: manual group-based validation, stronger augmentation, BatchNorm, higher dropout, stronger weight decay, label smoothing, early stopping, best-checkpoint selection, transfer learning, and low-learning-rate fine-tuning. Despite these measures, the gap remained.

A likely explanation is the structure of the dataset. Consecutive video frames can be nearly identical, while a manually separated validation group may contain different subjects, poses, backgrounds, or recording conditions. In addition, the three posture labels can differ through relatively subtle body geometry. Augmentation can vary crop, angle, color, and occlusion, but it cannot create genuinely new people and recording environments.

A confusion matrix was produced for an earlier MobileNet diagnostic run, but it corresponded to a different 46.34% experiment and was therefore not reused as if it belonged to the final 48.81% or 50.57% models. This report avoids mixing metrics from incompatible split protocols.

5.6 Experimental Development Process

Development Step	Reason	Observed Outcome
Custom CNN baseline	Establish a from-scratch reference.	35.50% final test accuracy; severe class/generalization problems.
Stronger augmentation and regularization	Reduce memorization and overconfidence.	Training remained strong, but validation instability persisted.
MobileNetV3 transfer learning	Reuse pretrained visual features.	Test accuracy increased to 48.81%.
Fine-tune last 4 feature blocks	Adapt pretrained features to squat-specific cues.	Best result increased to 50.57%, but overfitting remained.

6. Conclusion

This project developed and compared three stages of a squat-posture image classification pipeline: a custom CNN, MobileNetV3 transfer learning, and fine-tuned MobileNetV3. The final test accuracy improved from 35.50% to 48.81% and finally to 50.57%. The main positive result is that pretrained features clearly improved performance over the custom CNN, and selective fine-tuning provided an additional gain.

The main limitation is generalization. Very high training accuracy together with low validation performance shows that the models still learned training-specific patterns. Future work should prioritize collecting more diverse subjects, camera angles, clothing, backgrounds, and recording sessions; revisiting split design; systematic hyperparameter tuning; and evaluating pose/keypoint-based representations. Grad-CAM could also be added to inspect whether predictions are based on relevant body regions rather than background shortcuts.

7. References

- [1] CENG 476 - Introduction to Deep Learning, Homework Assignment: Deep Learning Project, Summer Term 2025-2026.
- [2] PyTorch and torchvision documentation (framework and pretrained model implementation used in the project).
- [3] A. Howard et al., "Searching for MobileNetV3," Proceedings of ICCV, 2019.
- [4] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," ICLR, 2019.